SYNRC NITRO/FORM

A TAD-native X-Forms Framework
for BTRON3 SPEC 3.20

Synrc / B-TRON Cleanroom Working Group

2026

Contents

1 Introduction

SYNRC NITRO/FORM is a lightweight, document-centric forms and business-application framework designed as a *natural and legal extension* of BTRON3 SPEC 3.20. It treats a business form as a first-class TAD real object and builds all higher-level structure on top of the existing Window Manager, Panel Manager, Parts Manager, Display Primitives (DP), and the Real/Virtual Object model.

The framework deliberately avoids any modification of core system calls, data structures, or standard TAD segment definitions. All new functionality is expressed through:

- application-range fusen and segments,
- ordinary application-level libraries,
- registered virtual-object / data-type handlers.

NITRO/FORM therefore remains 100 % compatible with any conforming BTRON3 SPEC 3.20 implementation (including the B-System / Synrc cleanroom).

2 Design Goals

1. Forms are ordinary TAD documents and participate fully in the / hypertext model.
2. Zero breakage of SPEC 3.20 APIs.
3. Simple, predictable structure that maps cleanly to both native rendering and limited HTML5 interchange.
4. Support for classic business data-entry applications (X-Forms style) while remaining lightweight.
5. Extensibility for custom renderers, layout engines, and semantic components without changing the core.

3 Core Abstraction — The Form

A **Form** is defined as a single TAD real object (or a well-formed TAD fragment) with exactly three logical regions:

Region	Content	Purpose
HEADER	TITLES	One or more title / subtitle strings
BODY	FIELDS	Ordered list of Field tuples
FOOTER	BUTTONS	Ordered list of action buttons

3.1 Field Tuple

$$Field = (STRINGLABEL,\ UICONTROL)$$

- **STRING LABEL** — human-readable label (TRON Code text).
- **UI CONTROL** — reference to a control managed by the Parts Manager or by the higher-level NITRO UI Control library (text box, numeric box, switch, selector, etc.).

This structure is deliberately minimal yet sufficient for the majority of business forms.

4 TAD Realisation

The Form is stored as a standard TAD document using only legal mechanisms:

```
Form TAD
          TS_INFO
          TS_FIG                    % form canvas / coordinate space
                HEADER region
                      title text segments + optional text fusen
                BODY region
                      Field 1  (label text + control descriptor fusen)
                      Field 2  ...
                      ...
                FOOTER region
                    button descriptors (Parts or higher-level buttons)
          optional data-instance record (current field values)
```

- Layout coordinates or a simple layout descriptor may be stored in an application-range fusen.
- Control identity, initial value, validation rules and action identifiers are also carried in application fusen.
- Unaware applications simply skip the application fusen and still see the visible text and graphics — exactly as TAD was designed.

5 Architecture Overview

```
Business Application


NITRO/FORM Library
  • Form parser / serialiser
  • Field & Button management
  • Simple layout engine
  • Data binding & validation
  • Event dispatch


SPEC 3.20 Public Services
  • Parts Manager   (UI controls)
  • Panel / Window Manager
  • Display Primitives (DP)
  • TAD + /
  • µITRON tasks (optional background work)
```

All drawing is performed through official Drawing Environments (). All interactive controls are ordinary Parts (or thin wrappers around them).

6 UI Control Layer

NITRO/FORM builds a thin, higher-level control library on top of the existing Parts Manager:

- Label + Text / Numeric / Secret box
- Check-box / Radio group

- Combo / Scroll selector
- Button (text or pictogram)
- Simple table / repeating group (dynamic creation of field rows)

Composite controls are created by composing standard Parts and, when necessary, custom-drawn regions using DP. No new system-level control types are introduced.

7 Layout and Custom Rendering

SPEC 3.20 provides no automatic high-level layout manager. NITRO/FORM therefore supplies a *simple, application-level* layout engine:

- Vertical stack (default for BODY fields)
- Two-column (label — control)
- Basic grid for tables
- Absolute / relative positioning when required

Custom renderers are ordinary application code that obtain a window's Drawing Environment and issue DP calls. A Form handler may also embed arbitrary TAD figure data or invoke an optional PostScript/Forth-style interpreter for advanced graphics, still without touching core APIs.

8 Semantic Mapping (HTML5-inspired)

For interchange and documentation the following natural mapping is defined:

HTML5-inspired element	NITRO/FORM / TAD realisation
`article` / form container	top-level Form TAD
`header`	HEADER region
`section`	nested figure or field group
`aside`	side panel (Panel Manager)
`nav`	FOOTER buttons or navigation bar of virtual objects
`table`	grid of Fields or TAD figure table
`picture` / `img`	`TS_IMAGE` or linked real object
`form` controls	Field tuples (LABEL + UI CONTROL)

Bidirectional conversion between a useful HTML5 subset and a NITRO/FORM TAD document is supported by the companion TAD Browser component.

9 Runtime Lifecycle

1. Open Form TAD (or receive a virtual object pointing to it).
2. Parse HEADER / BODY / FOOTER regions.
3. Instantiate UI CONTROLS via Parts Manager (or NITRO control library).
4. Perform layout and draw titles, fields and buttons into the window's Drawing Environment.
5. Bind field values to a data instance (another TAD record or in-memory model).
6. Enter event loop: Parts events + window redraw requests are dispatched by the library.
7. On save / submit the current control values are written back into the TAD (or a linked data real object).

10 Data Binding and Validation

- Each Field may declare a simple type, required flag, range, or regular expression (stored in fusen).
- Calculated fields and relevance (show/hide) are evaluated by the library.
- The data instance can itself be a TAD document, preserving the / model.

11 Extensibility Points

- New control types — implemented as library composites or custom DP drawing.
- Advanced layout engines — replace the simple layout module.
- Custom renderers — any code that draws into a Drawing Environment.
- Media (`video/audio`) — application fusen + handler.
- Scripting — optional embedding of a PostScript/Forth subset for dynamic behaviour.

All extensions remain application-level and therefore legal under SPEC 3.20.

12 Relationship to Existing BTRON Components

- **Parts Manager** — supplies the atomic UI CONTROLS.
- **Panel / Window Manager** — provides the surfaces on which Forms are displayed.
- **Display Primitives** — used for all custom drawing and title rendering.
- **TAD + /** — persistence, hyperlinking, and document-centric nature.
- **TAD Browser** (companion) — on-the-fly HTML ↔ NITRO/FORM conversion for documentation.

13 Conclusion

SYNRC NITRO/FORM demonstrates that a practical, modern X-Forms-style framework can be realised as a pure extension of BTRON3 SPEC 3.20. By defining a Form as a TAD document with the simple triple structure

$$\text{HEADER of TITLES} \quad + \quad \text{BODY of (LABEL, CONTROL) tuples} \quad + \quad \text{FOOTER of BUTTONS}$$

and by building exclusively on public managers and Drawing Environments, the framework stays completely compatible with the original specification while enabling productive business-application development in the native BTRON environment.